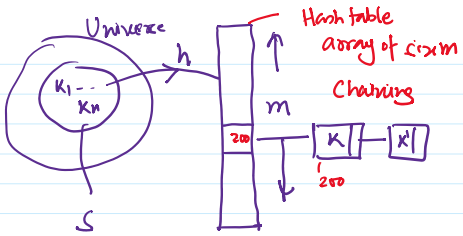
Lecture 7 Open Addressing

11 August 2026 10:55

- In open addressing, all elements occupy the hash table itself.
- That is, each table entry contains either an element of the dynamic set or NIL.
- When searching for an element, we systematically examine table slots until either we find the desired element or we have ascertained that the element is not in the table.
- No lists, no elements are stored outside the table, unlike in chaining.



$h \in \mathcal{H}$ Universal Hash family

$m < n$

$\exists \; K \neq K'$ s.t $h(K) = h(K')$

Collision

Open add. Hash table – array of struct

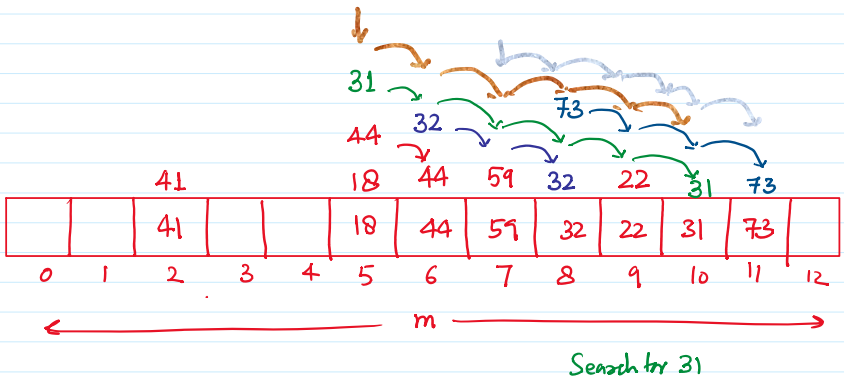
Linear Probing

We store (Key, val) pair in hash table

Data :=(Key, Value)

$h(K) = K \bmod 13$

Insert keys 18, 41, 22, 44, 59, 32, 31, 73



Search for 31

Insert: Linear Probe Insert (K)

Compute 31 mod 13

Search for 33

```
if (table is full)
    error
probe = h(K)
while (table(probe) occupied)
    probe = (probe + 1) mod m
table[probe] ← K
```

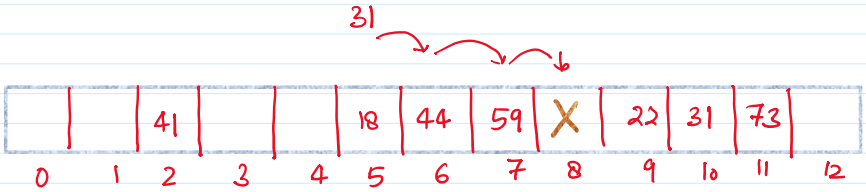
Search in linear Probing

- To search for a key K, we go to (K mod 13), and continue looking at successive location until we find K, or encounter with empty location
- Successful Search: To search for 31, compute (31 mod 13) = 5 and continue location 6, 7, -- till we find 31 at location 10

- Unsuccessful Search: To search 33, compute 33 mod 13 = 7, continue till we encounter the empty location

Deletion in Linear Probing : Suppose we need to delete element 32.

32 mod 13



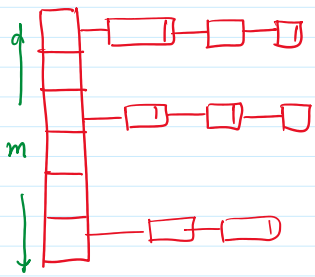
- to delete 32 we first search for 32
- 32 is found at location 8. Suppose we set this location to NULL.
- Suppose we want to search for 31. we will encounter NULL before seeing 31

⇒ 31 is not present ✗

- Instead of setting location 8 to NULL, we place a marker X.
- During Search when we see X, ignore it and continue to next location
- During insert if X came across, we put that element on the location & remove X.
- Too many X degrades search performance
- Rehash if there are too many markers X

Hash table based techniques for dictionary Problem

Collision By chaining

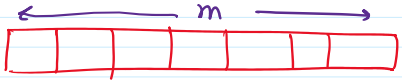


load factor

↓

$\frac{n}{m} := \alpha$

Linear Probing



$m > n$

expected search time $O(1+\frac{n}{m}) = O(1+\alpha)$

$n > m$

m size of hash table

expected search time $O(1+\frac{n}{m}) = O(1+\alpha)$

$n > m$

$\alpha > 1$

m size of hash table

n #of keys

when key is present in hash table : Sucessful search

" is not present " " : Unsuccessful Search

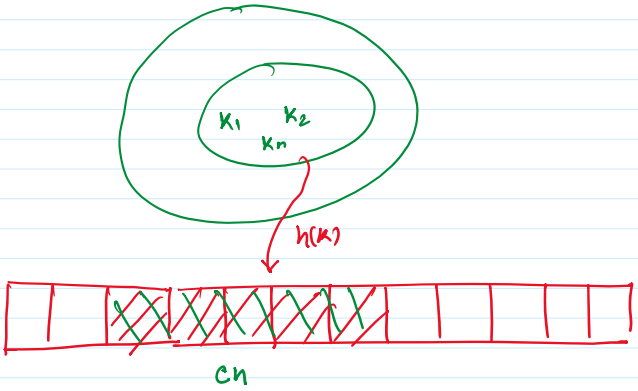
In Collision By chaining

Expected Search time — Sucessful Search — $O(1+\alpha)$

Unsuccessful Search

$\alpha = \frac{n}{m}$

- linear Probing is slower than chaining
- linear probing use less memory as one doesn't need to store ptr used in linked list.



Double Hashing

- Use two hash function h_1 & h_2
- $h_1(K)$ gives the position of key K in hash table
- $h_2(K)$ gives a step count for key K.

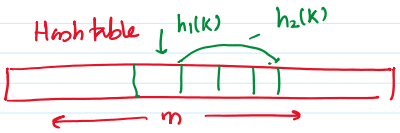
n #of key

m # slots in Hashtable

$n < m$

- $h_1(K)$ gives the position of key K in hash table — m # slots in Hash table
- $h_2(K)$ gives a step count for key K. — $n < m$
- In linear probing $h_2(K) = 1$

$h_1(K) \in \{0 \ldots m-1\}$
$h_2(K) \in \{0 \ldots m-1\}$



Double Hashing Insert (K)

if (table is full) error

probe ← $h_1(K)$, offset ← $h_2(K)$

while table[probe] is occupied

{ probe ← (probe + offset) mod m }

table[probe] ← K

$h_2(K)$ must be relatively prime to m
⇒ every location of hash table is accessible

load factor := $\alpha = \frac{n}{m} < 1$

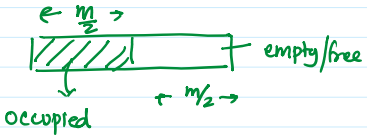
[Search, Delete is same as linear probing + using $h_2(K)$ th as offset]

h_1, h_2 are universal Hash fn

[n: # of keys
m: # of slots in hash table
$n < m$]

- ✓Assume that final effect of h_1 - h_2 gives a random location in hash table.
- After hashing n keys, (m-n) slots are empty ⇒ $\left(\frac{m-n}{m}\right)$ fraction of table is empty

⇒ $(1-\alpha)$ fraction of table is empty $\boxed{\alpha = \frac{n}{m}}$



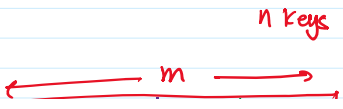
$\alpha = 1/2$

- Expected # of probes/trails required to find an empty location is $\frac{1}{(1-\alpha)}$

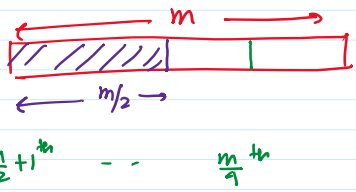
$\bar{K}$ (is not present in hash table) = unsuccessful search = $\frac{1}{(1-\alpha)}$

To insert an element we need to find an empty location

Inserting	Avg no. of probes	Exp Total no. of probes
First m/2	≤ 2	$\leq m$
Next m/4	≤ 4	$\leq m$
Next m/8	≤ 8	$\leq m$
⋮	⋮	⋮
Next $m/2^i$	$\leq 2^i$	$\leq m$



n keys



No. of probes required to insert

$\underbrace{\frac{m}{2} + \frac{m}{4} + \cdots + \frac{m}{2^i}}$ elements = # of required to leave $\frac{1}{2^i}$ fraction of table empty

$= mi$

$\frac{k}{=}$

p

$\frac{1}{2^i} = 1-\alpha$

$\Rightarrow i = -\log(1-\alpha)$

#of probes required to leave $(1-\alpha)$ fraction of table empty $= -m\log(1-\alpha)$

$\Rightarrow$ Avg #of probes required to insert n elements is $= -\frac{m}{n}\log(1-\alpha)$

$= -\frac{1}{\alpha}\log(1-\alpha)$

$= \frac{1}{\alpha}\log\left(\frac{1}{1-\alpha}\right)$

$\boxed{\alpha = \frac{n}{m}}$

	Unsuccessful Search	Successful Search
Chaining	$O(1+\alpha)$	$O(1+\alpha)$
Probing	$O\left(\frac{1}{1-\alpha}\right)$	$O\left(\frac{1}{\alpha}\log\left(\frac{1}{1-\alpha}\right)\right)$